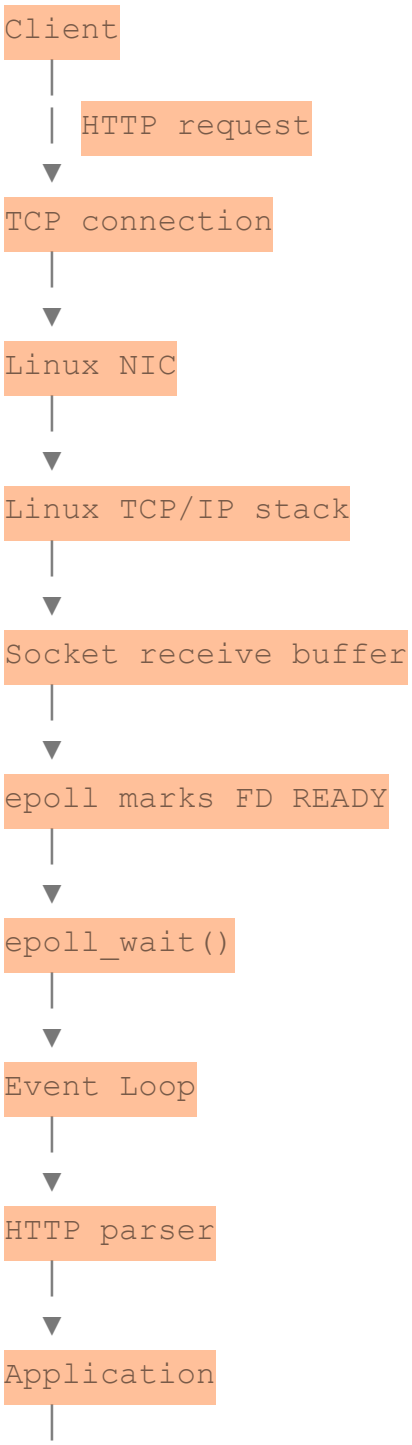
Absolutely. The easiest way to understand `epoll` deeply is to trace **one HTTP request from the client all the way through the Linux kernel, `epoll`, the event loop, application code, and back to the client**.

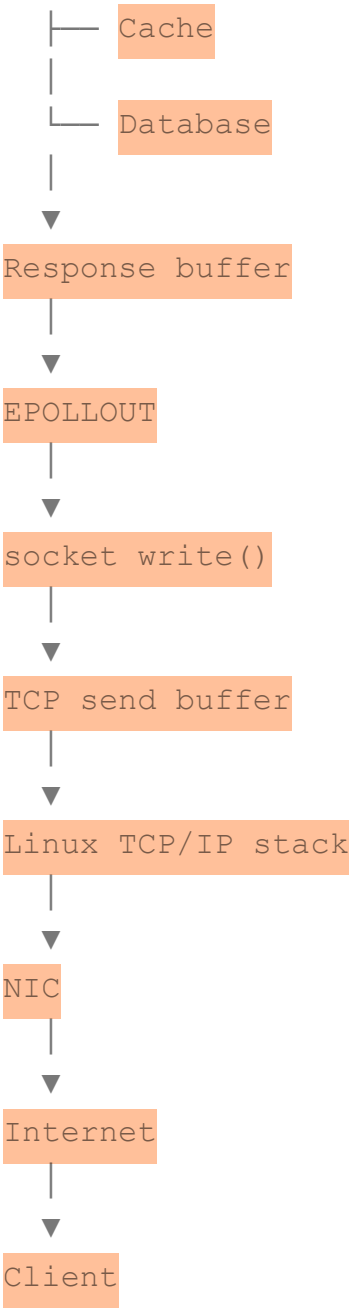
I'll show both **request** and **response** flows, then the **ET/LT differences**, **partial reads/writes**, and finally the **multi-core production architecture**.

---

1. Complete end-to-end request/response

Assume:





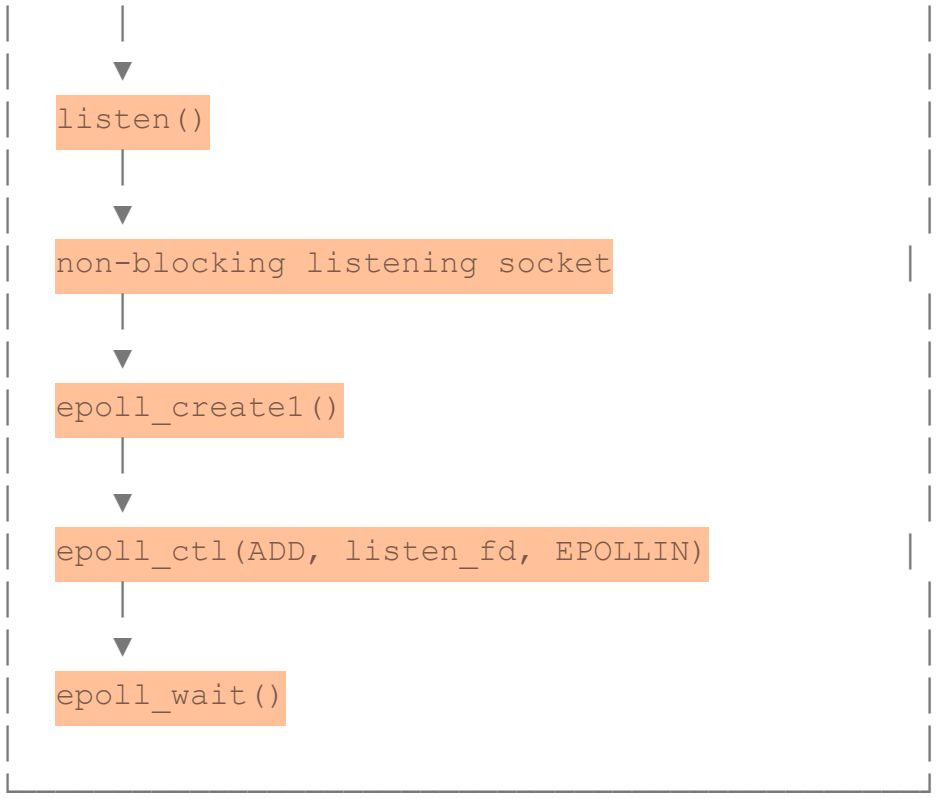
Now let's go through it carefully.

---

2. Step 0 — Server initialization

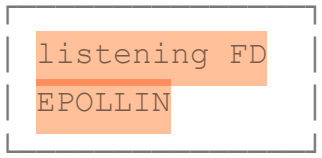
Before any request arrives, the server establishes its event loop.





At this point:

```
epoll interest list
```



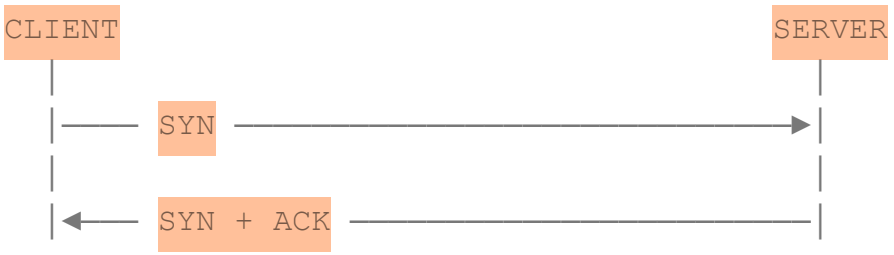
The server is essentially saying:

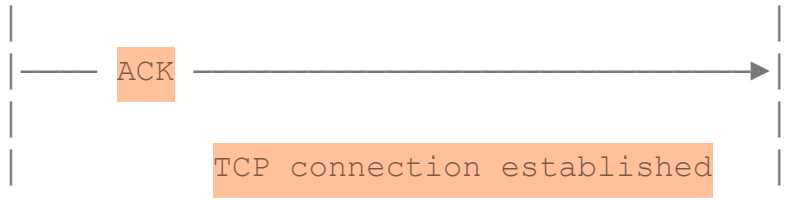
> "Wake me when a new connection can be accepted."

---

3. Step 1 — TCP connection arrives

A client initiates a TCP connection.

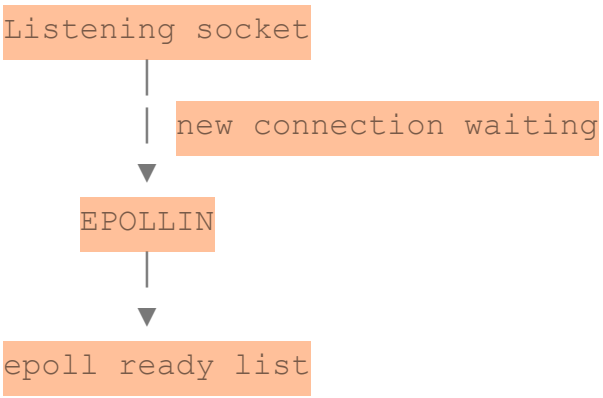




The Linux kernel now has a connection associated with the listening socket.

The listening socket becomes readable from the server's perspective.

Conceptually:



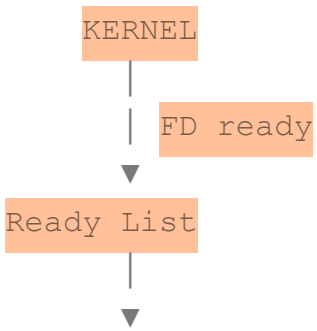
---

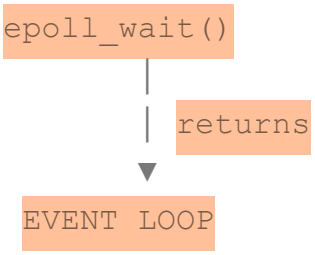
4. Step 2 — `epoll_wait()` wakes up

The event loop was sleeping:

```
while (true) {
    events = epoll_wait(...);
    ...
}
```

The kernel wakes it:





The application receives something like:

```
event.fd = listen_fd
event.events = EPOLLIN
```

---

5. Step 3 — Server calls `accept()`

The event loop recognizes:

```
event.fd == listen_fd
```

and performs:

```
accept()
```

The kernel creates a new connected socket:

```
listen_fd = 10

accept()
   ↓
client_fd = 27
```

Now:

```
FD 10 → listening socket
FD 27 → client connection
```

---

6. Step 4 — Configure the client socket

The server sets:

```
FD 27
    │
    ├── O_NONBLOCK
    │
    └── protocol state
```

Then registers it:

```
epoll_ctl(
    EPOLL_CTL_ADD,
    27,
    EPOLLIN
)
```

The conceptual interest list is now:

```
┌──────────────────────────────┐
│ epoll interest list          │
├──────────────────────────────┤
│ FD 10 → EPOLLIN              │ ← listening socket
│ FD 27 → EPOLLIN              │ ← client connection
└──────────────────────────────┘
```

The server goes back to:

```
epoll_wait()
```

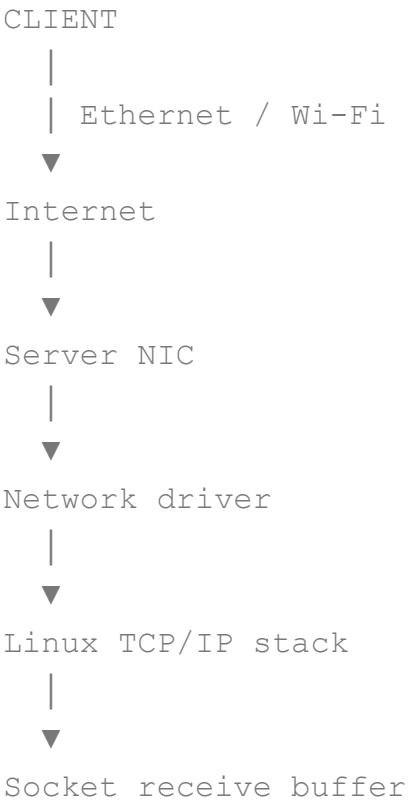
---

7. Step 5 — Client sends HTTP request

Suppose the client sends:

```
GET /users/123 HTTP/1.1
Host: example.com
Connection: keep-alive
```

The network path is approximately:



The important part is:



---

8. Step 6 — Kernel marks FD readable

The kernel recognizes:



has data available.

Conceptually:



```
EPOLLIN
   |
   ▼
epoll ready list
```

So:

```
Interest list                Ready list

FD 10 — EPOLLIN              FD 27
FD 27 — EPOLLIN      ⟶
```

Notice the important distinction:

Interest list:

> What am I interested in?

Ready list:

> What is ready right now?

---

9. Step 7 — `epoll_wait()` returns

The event loop wakes:

```
epoll_wait()
     |
     ▼
┌──────────────┐
| FD 27        |
| EPOLLIN      |
└──────────────┘
```

Then:

```
dispatch(FD 27)
```

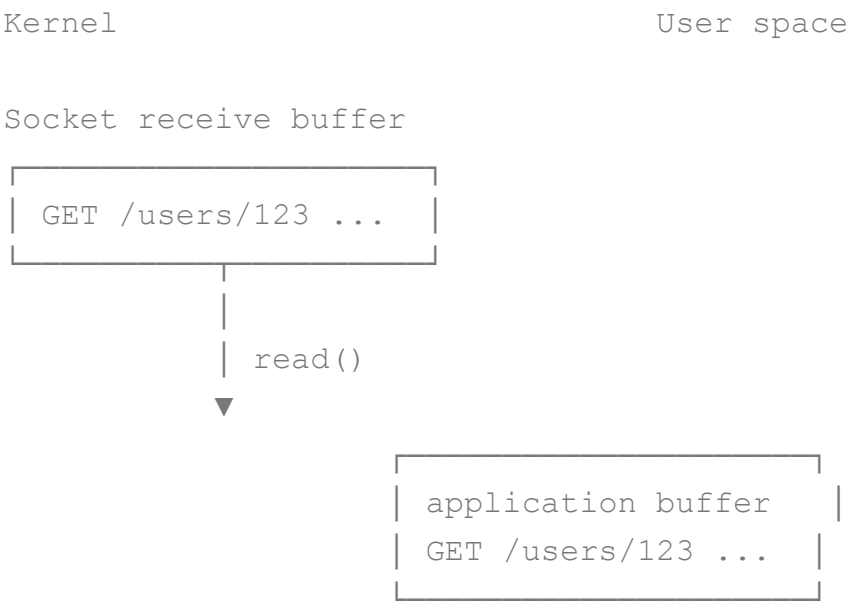
---

10. Step 8 — Application performs `read()`

The event loop calls:

```
read(27, buffer, ...)
```

The kernel copies data:



Now the application owns the bytes.

---

11. Important: TCP doesn't preserve HTTP messages

Suppose the client sends:

```
GET /users/123 HTTP/1.1
```

The server might receive:

```
GET /users/
```

on the first `read()`.

Then:

```
123 HTTP/1.1
```

on the second.

Or everything could arrive in one read.

Or multiple HTTP requests could arrive in a single TCP read.

Therefore:

```
TCP
 ↓
byte stream
```

not:

```
TCP
 ↓
HTTP messages
```

The application needs an HTTP parser and buffering.

---

12. Step 9 — HTTP parser

The application maintains:

```
Connection
 ├── readBuffer
 ├── writeBuffer
 ├── parserState
 └── requestState
```

For example:

```
readBuffer

┌──────────────────────────────────────┐
│ GET /users/123 HTTP/1.1              │
│ Host: example.com                    │
│ Connection: keep-alive               │
│                                      │
└──────────────────────────────────────┘
```

The parser converts bytes into:

```
HTTP Request

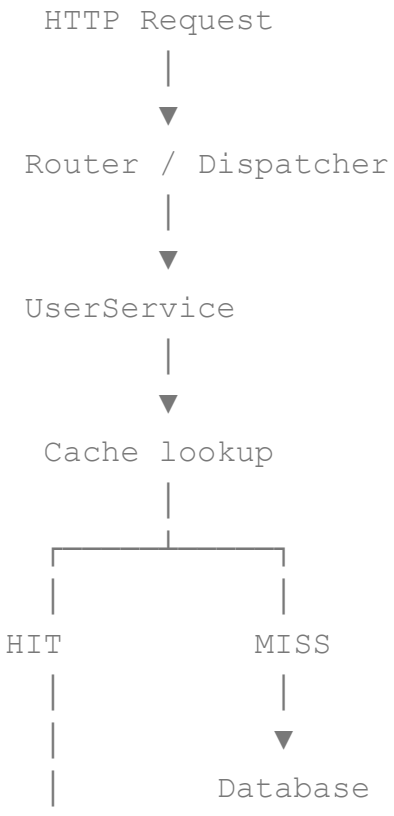
method = GET
path = /users/123
headers = ...
```

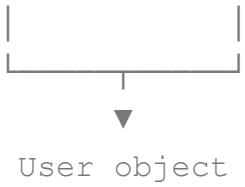
---

13. Step 10 — Application processing

Now the network layer is mostly done.

The request flows into the application:





For example:

```
GET /users/123
       |
       ▼
UserController
       |
       ▼
UserService
       |
       ▼
Redis
       |
       ▼
User
```

---

14. Step 11 — Build HTTP response

The application generates:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 48
Connection: keep-alive

{
  "id": 123,
  "name": "Alice"
}
```

But the response is **not necessarily written directly to the socket**.

Instead:

```
Application
     |
     ▼
writeBuffer
```

Conceptually:

```
Connection 27

writeBuffer
┌──────────────────────────────────────┐
│ HTTP/1.1 200 OK\r\n                  │
│ Content-Type: application/json\r\n   │
│ ...                                  │
│ {"id":123,"name":"Alice"}            │
└──────────────────────────────────────┘
```

---

15. Step 12 — Enable `EPOLLOUT`

Now the server has data to send.

It changes the interest registration:

```
epoll_ctl(
    EPOLL_CTL_MOD,
    27,
    EPOLLIN | EPOLLOUT
)
```

Why?

Because the server is saying:

> "Notify me when FD 27 can accept more outgoing data."

So:

```
Before:

FD 27 → EPOLLIN
```

```
After:

FD 27 → EPOLLIN | EPOLLOUT
```

---

16. Step 13 — Socket becomes writable

TCP's send buffer has capacity.

The kernel reports:

```
EPOLLOUT
```

Conceptually:



The event contains:

```
FD 27
EPOLLOUT
```

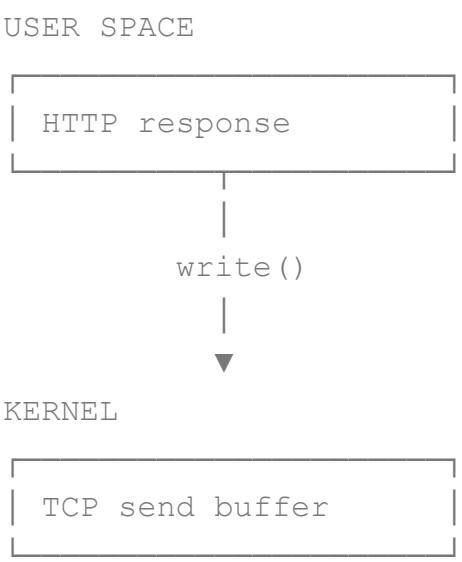
---

17. Step 14 — Application calls `write()`

The event loop executes:

```
write(27, writeBuffer, ...)
```

Data moves:



Then the TCP stack transmits it.

---

18. Partial write

Suppose:

```
writeBuffer = 10 MB
```

but the kernel accepts:

```
64 KB
```

Then:

```
10 MB
 │
 ├── 64 KB → kernel
 │
 └── ~9.94 MB remains
```

The application must preserve the remaining data.

```
writeBuffer
```

```
┌──────────────────────────────┐
│ remaining 9.94 MB            │
└──────────────────────────────┘
```

It keeps:

```
EPOLLOUT
```

enabled.

---

19. Continue writing

Eventually:

```
write()
 ↓
64 KB
 ↓
write()
 ↓
128 KB
 ↓
write()
 ↓
...
```

until:

```
writeBuffer = EMPTY
```

Then:

```
epoll_ctl(
    MOD,
    FD 27,
```

```
    EPOLLIN
)
```

Notice:

```
EPOLLOUT disabled
```

This avoids unnecessary writable notifications.

---

20. Response reaches client

The final path:

```
USER SPACE
    |
    | write()
    ▼
KERNEL
    |
    ▼
TCP send buffer
    |
    ▼
TCP/IP stack
    |
    ▼
NIC
    |
    ▼
Internet
    |
    ▼
Client NIC
    |
    ▼
Client TCP stack
    |
    ▼
Client application
    |
```

```
    ▼
HTTP response
```

So the complete flow is:

```
REQUEST
═══════════════════════════════════════════════════════════════

Client
  │
  ▼
NIC
  │
  ▼
Linux TCP/IP
  │
  ▼
Socket RX buffer
  │
  ▼
epoll → EPOLLIN
  │
  ▼
epoll_wait()
  │
  ▼
Event Loop
  │
  ▼
read()
  │
  ▼
HTTP Parser
  │
  ▼
Application
  │
  ▼
Cache / DB
  │
  ▼
Response
```

```
RESPONSE
═══════════════════════════════════════════════════════════════

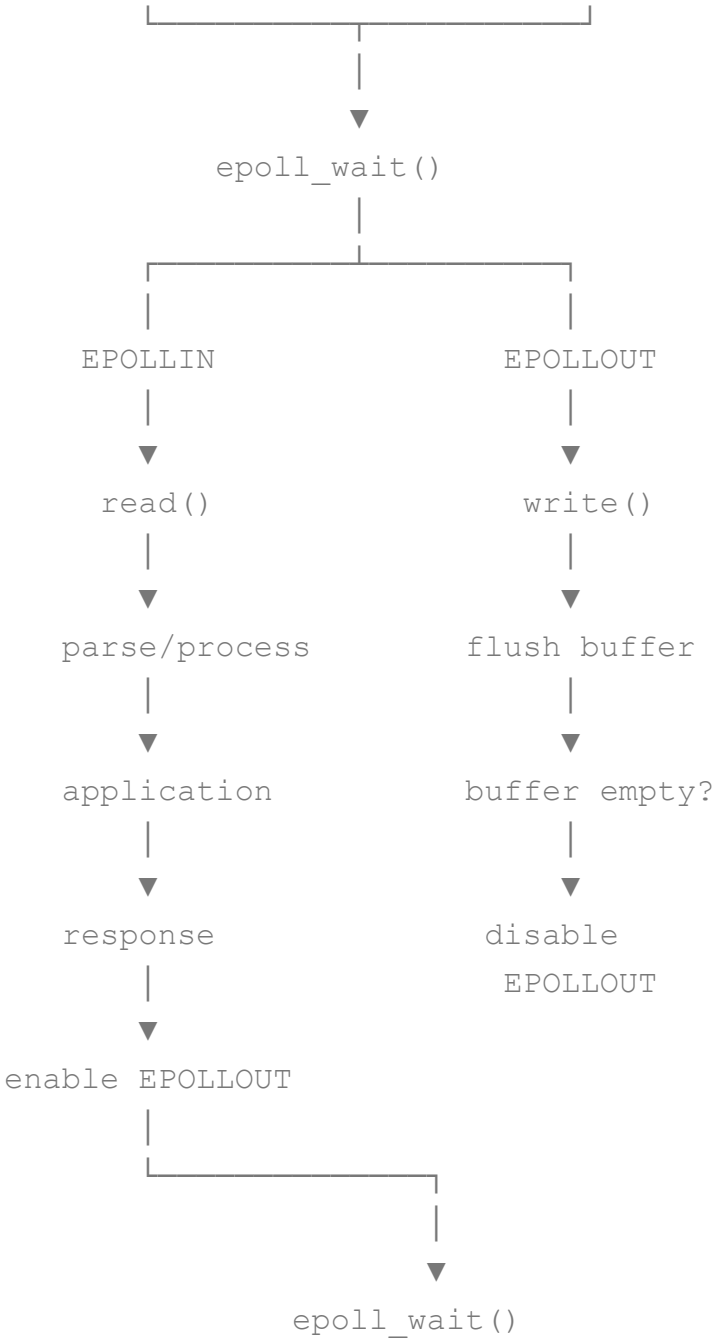
Response
  │
  ▼
Write Buffer
  │
  ▼
epoll → EPOLLOUT
  │
  ▼
epoll_wait()
  │
  ▼
Event Loop
  │
  ▼
write()
  │
  ▼
TCP TX buffer
  │
  ▼
TCP/IP
  │
  ▼
NIC
  │
  ▼
Internet
  │
  ▼
Client
```

---

21. The event-loop view

From the server's perspective, the entire process is essentially:

```
┌──────────────────────────┐
│        Event Loop        │
```



This is the **Reactor pattern** in action.

---

22. Edge-triggered version

Now consider:

```
EPOLLIN | EPOLLET
```

The rule changes.

Suppose:

```
Kernel RX buffer

┌──────────────────────────────┐
│ 100 KB available             │
└──────────────────────────────┘
```

You receive one event:

```
EPOLLIN
```

Then:

```
read()
→ 32 KB
```

Remaining:

```
68 KB
```

With edge-triggered mode, you should **not wait for another event**.

Instead:

```
read()
read()
read()
read()
...
```

until:

```
read()
→ EAGAIN
```

Illustration:

```
EPOLLIN
   │
   ▼
```

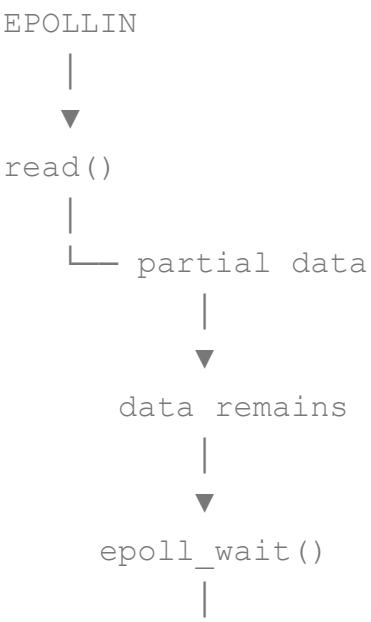


This is why ET requires careful non-blocking I/O.

---

23. Level-triggered version

With the default level-triggered behavior:



```
          ▼
       EPOLLIN again
```

So:

```
LT
→ "You still have work to do."

ET
→ "The readiness state changed; you need to drain it."
```

A useful simplified mental model.

---

24. Keep-alive request flow

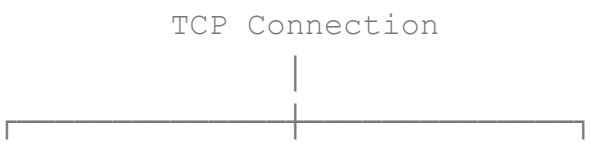
HTTP/1.1 commonly keeps the TCP connection open.

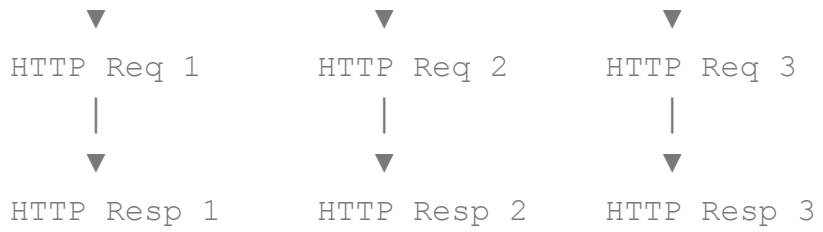
Therefore:

```
Request 1
   ↓
Response 1
   ↓
Request 2
   ↓
Response 2
   ↓
Request 3
   ↓
Response 3
   ↓
...
```

There is **no new TCP handshake for every request**.

Architecture:





This is particularly important for understanding why servers may have:

```
1 million TCP connections
```

but potentially many more HTTP requests over those connections.

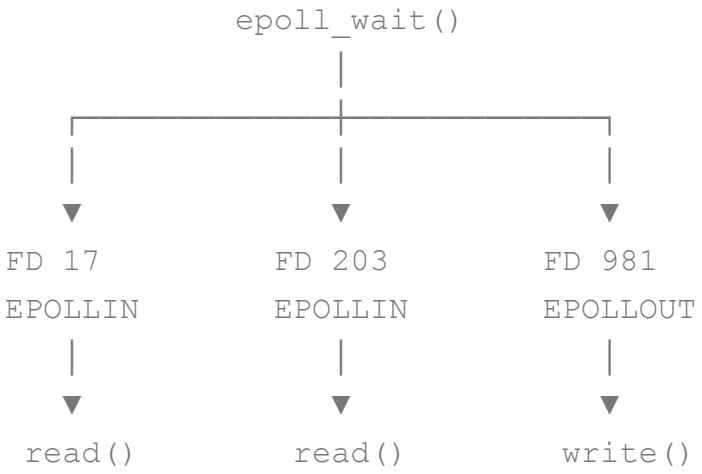
---

25. Multiple connections

Now imagine:

```
10,000 TCP connections
```

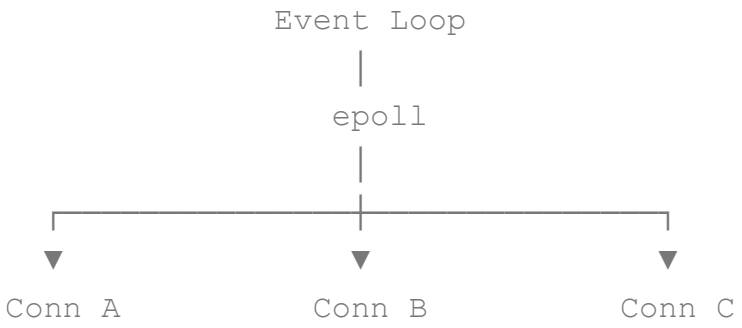
The event loop looks like:



The server does **not** need:

```
Thread 1  → FD 17
Thread 2  → FD 203
Thread 3  → FD 981
...
```

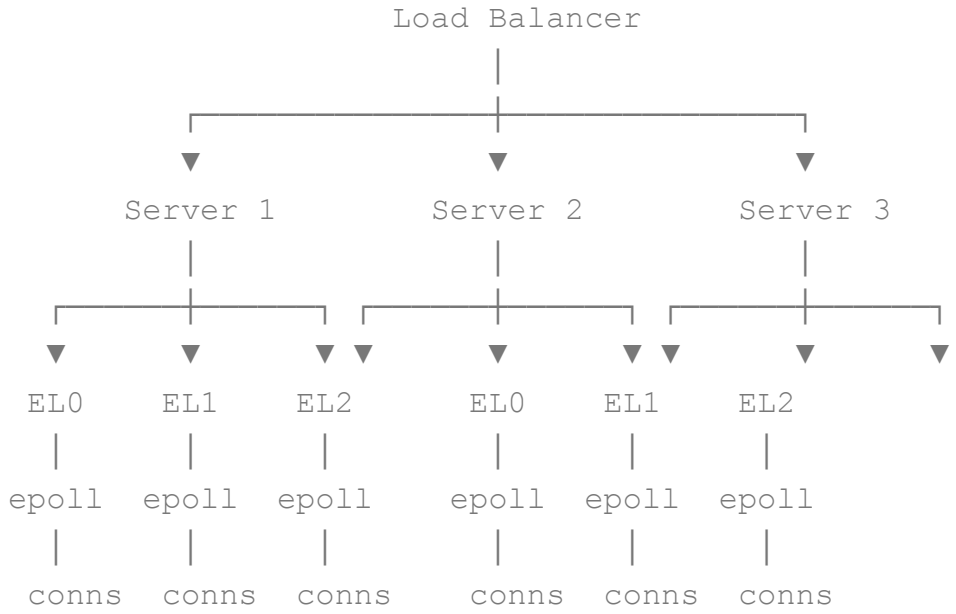
Instead:



---

26. Multi-core production server

A high-performance server might look like:



Where:

```
EL = Event Loop
```

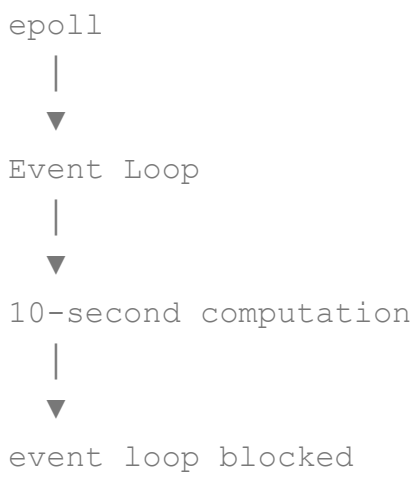
Each event loop can be associated with a CPU/core.

---

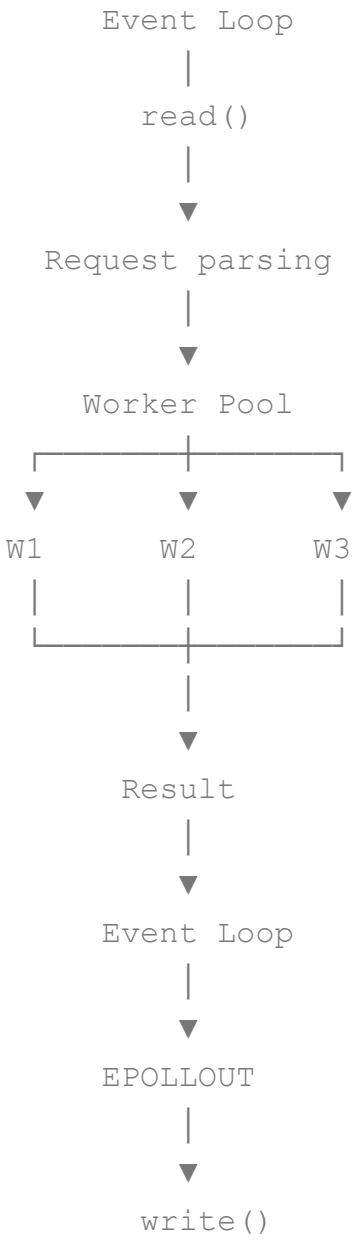
27. Where worker threads fit

Suppose an HTTP request requires CPU-heavy work.

Don't do this:



Instead:



This keeps the event loop responsive.

28. One subtle but important point

`epoll` doesn't mean:

```
"the kernel sends the request to the application."
```

Instead:

```
kernel
  │
  │ records readiness
  ▼
epoll
  │
  │ tells application
  ▼
application
  │
  │ explicitly performs
  ▼
read()
```

Likewise for responses:

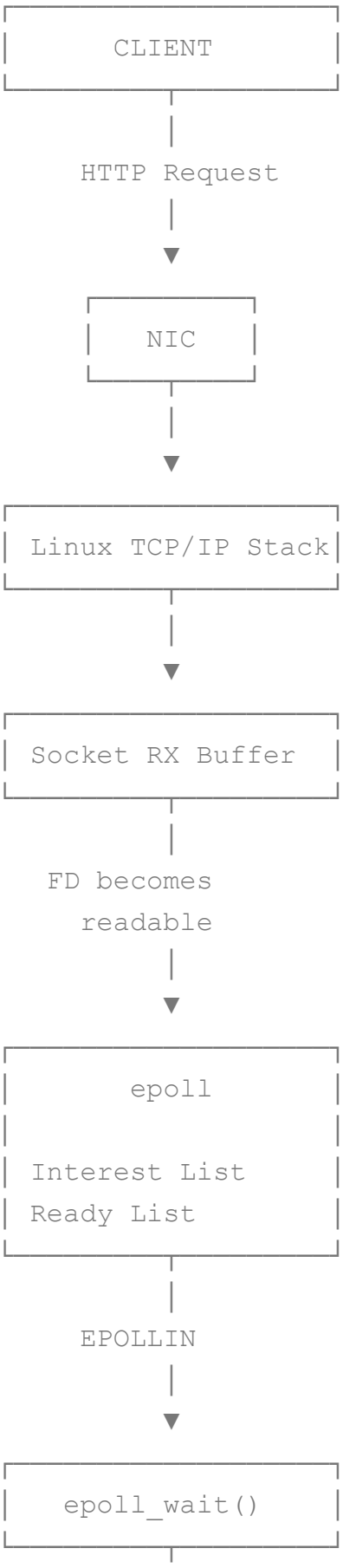
```
application
  │
  │ explicitly performs
  ▼
write()
  │
  ▼
kernel
```

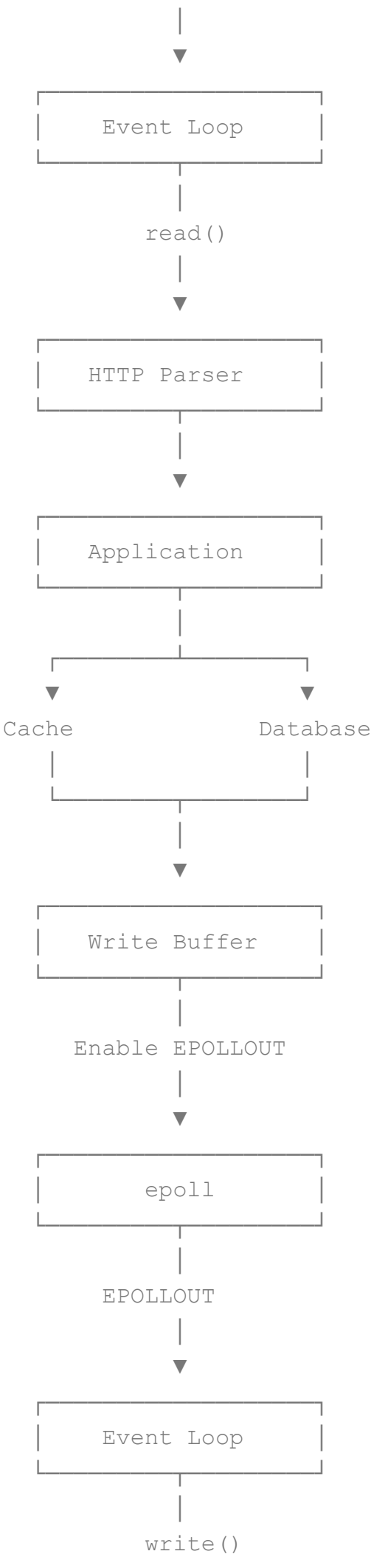
So:

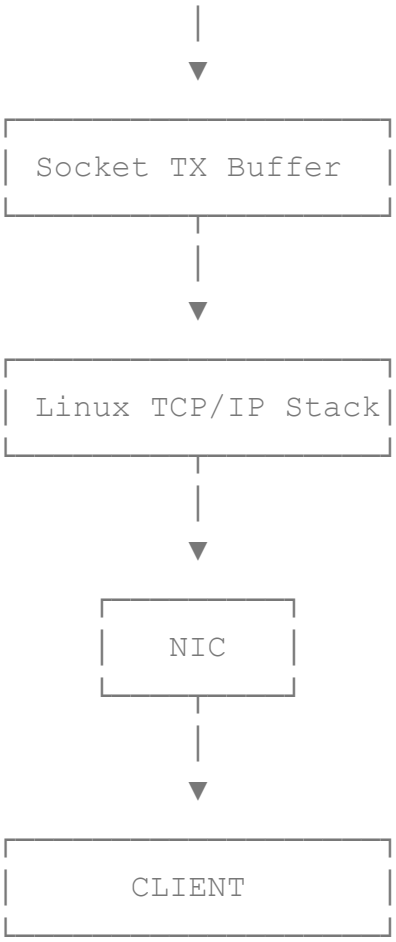
> **`epoll` is a readiness notification mechanism sitting between the application's event loop and the kernel's I/O subsystem.**

29. The entire picture in one diagram

This is the diagram I'd memorize for interviews:




Event Loop
read()
HTTP Parser
Application
Cache
Database
Write Buffer
Enable EPOLLOUT
epoll
EPOLLOUT
Event Loop
write()

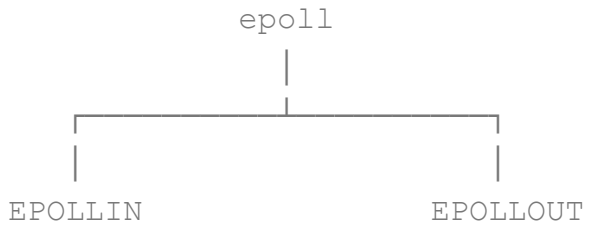


The five transitions to remember

For interviews, reduce the entire diagram to these five transitions:

```
1. Network → Socket Buffer
        ↓
2. Socket Buffer → epoll readiness
        ↓
3. epoll readiness → Event Loop
        ↓
4. Event Loop → read()/write()
        ↓
5. Kernel Socket → Network
```

And the most important conceptual distinction is:





That is the bridge from **Linux kernel internals → DSA → Reactor-pattern LLD → high-concurrency HLD**.